Modbus/TCP Attack Detection

Detection Rationale Report

Indurex — Assignment

Imane Lakdim

30 October 2026

1. Overview

This report covers the rationale behind the detection logic built for a simulated OT environment: a water tank controlled by an inlet pump and outlet valve, exposed over Modbus/TCP via OpenPLC. Four Linux/VirtualBox VMs form the lab an OpenPLC master, an OpenPLC slave acting as the plant, a Kali attacker, and a Wazuh + Zeek analysis host connected on an isolated host-only network. Five attack classes were executed against the plant, captured, converted with Zeek, and detected with custom Wazuh rules.

Full command by command detail, raw captures, and Zeek logs are kept in the accompanying repository; this document summarizes the build, the attacks, the detection rationale, and two honestly documented edge cases.

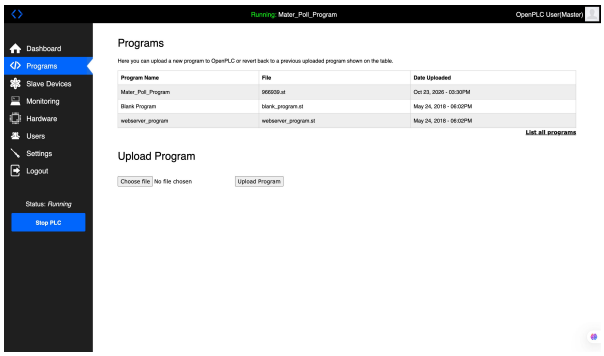
2. Environment Setup

2.1 Installation and programs

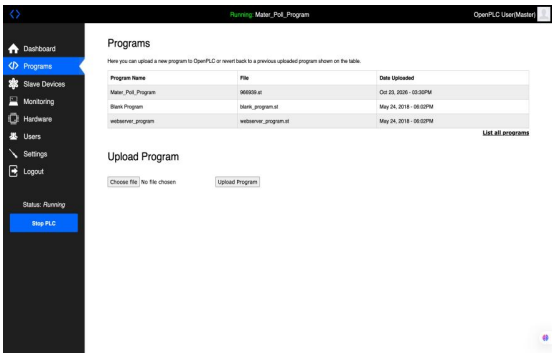
OpenPLC Runtime V3 was installed on two Ubuntu Server 22.04 VMs one as the Modbus/TCP master, one as the slave PLC using:

```
git clone https://github.com/thiagoralves/OpenPLC_v3.git cd OpenPLC_v3 ./install.sh linux
```

Two Structured Text programs were written and uploaded through each runtime's web interface: Tank_Control (slave) simulates the tank level evolution, pump control with hysteresis, and a hard coded 95% high-high alarm while Master_Poll (master) declares the remote variables it polls.



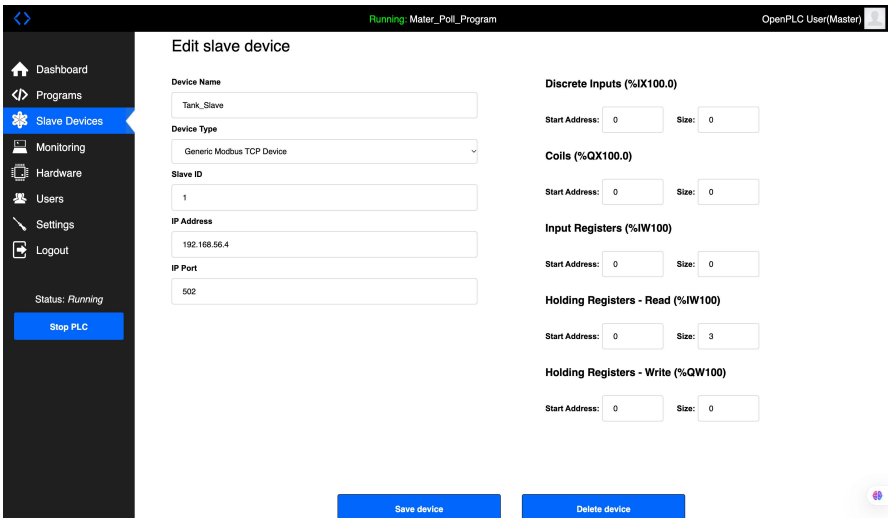
Slave runtime: Tank_Control compiled and running.



Master runtime: Master_Poll compiled and running.

2.2 Master–slave configuration and address map

The slave (192.168.56.4) was added as a remote device in the master's (192.168.56.3) web interface, with Modbus/TCP on port 502.



Slave Devices configuration on the master.

Type	Addr	Meaning	Normal range
Coil	0	Pump running	0 or 1
Coil	1	Outlet valve open	1 (default)
Coil	2	High-high alarm	0 (briefly 1 at ≥95%)
Hold. reg.	0	Tank level (%)	~19 – 81
Hold. reg.	1	High setpoint (%)	80 (design 70–90)
Hold. reg.	2	Low setpoint (%)	20 (design 10–30)

2.3 Baseline capture

A 5 minute baseline was captured on the slave with tcpdump, establishing the only legitimate traffic pattern: one source (the master), one function code (Read Holding Registers), one persistent connection, ~8 requests/second, no writes, no exceptions.

```
sudo tcpdump -i enp0s8 -n -w ~/baseline_normal.pcap port 502
```

```
slave@ubuntu:~$ tshark -r baseline_normal.pcap
    1   0.000000 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13536; Unit:   1, Func:   3: Read Holding R
egisters
    2   0.000144 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13536; Unit:   1, Func:   3: Read Holding R
egisters
    3   0.000877 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=13 Ack=16 Win=502 Len=0 TSval=4274901307 TSec
r=983417685
    4   0.101893 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13537; Unit:   1, Func:   3: Read Holding R
egisters
    5   0.102038 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13537; Unit:   1, Func:   3: Read Holding R
egisters
    6   0.102800 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=25 Ack=31 Win=502 Len=0 TSval=4274901409 TSec
r=983417787
    7   0.203752 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13538; Unit:   1, Func:   3: Read Holding R
egisters
    8   0.203853 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13538; Unit:   1, Func:   3: Read Holding R
egisters
    9   0.204478 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=37 Ack=46 Win=502 Len=0 TSval=4274901511 TSec
r=983417888
   10   0.305038 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13539; Unit:   1, Func:   3: Read Holding R
egisters
   11   0.305251 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13539; Unit:   1, Func:   3: Read Holding R
egisters
   12   0.305801 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=49 Ack=61 Win=502 Len=0 TSval=4274901612 TSec
r=983417990
   13   0.406279 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13540; Unit:   1, Func:   3: Read Holding R
egisters
   14   0.406380 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13540; Unit:   1, Func:   3: Read Holding R
egisters
   15   0.406969 192.168.56.3 → 192.168.56.4 TCP 66 53194 → 502 [ACK] Seq=61 Ack=76 Win=502 Len=0 TSval=4274901713 TSec
r=983418091
   16   0.508174 192.168.56.3 → 192.168.56.4 Modbus/TCP 78    Query: Trans: 13541; Unit:   1, Func:   3: Read Holding R
egisters
   17   0.508498 192.168.56.4 → 192.168.56.3 Modbus/TCP 81 Response: Trans: 13541; Unit:   1, Func:   3: Read Holding R
```

Baseline Modbus/TCP traffic inspected with tshark.

3. Attacks Executed

All five attack classes targeted the slave (192.168.56.4) from Kali (192.168.56.6). Several were run with both Metasploit and hand-written Python to compare how the same logical attack looks on the wire, which directly informed the rules in Section 4.

#	Attack	Mechanism	Effect on process
1	Reconnaissance	FC4 reads, unit-ID sweep, identification probe (Metasploit)	None read-only
2	Coil write	FC5, coil 1 (valve) forced closed	Tank froze: in/outflow both reached zero
3	Register write	FC6, high setpoint changed 80 → 100	Pump overran old limit; only a hard-coded, non-writable 95% alarm prevented overflow
4	Reset / diagnostic	Vendor UMAS stop (FC90) and standard Diagnostics (FC8)	Both rejected / no effect still independently detectable
5	Read flood	22,656 FC3 reads/15s on one connection; 123-packet variant over many short connections	None on process, network/device load only

Attack 3 is worth a specific note: the physical consequence was brief. The pump ran past the old 80% limit until the independent 95% alarm force stopped it but since the valve stayed open and kept draining, the alarm condition cleared within about one 100 ms PLC scan cycle. This was only confirmed by polling faster than the PLC's own scan rate, and it demonstrates why detecting the cause (the anomalous register write) is far more dependable than trying to catch the physical symptom.

4. Detection Pipeline and Rules

Captures were transferred from the slave to the Wazuh host via SCP, then converted to JSON logs with Zeek, run in Docker and pinned by image digest for reproducibility:

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v ~/modbus-assignment/pcaps:/pcaps:ro \
  -v ~/modbus-assignment/zeek-output/:/out \
  -w /out \

zeek/zeek@sha256:70733f4e540ba1608e37e00c6a93d009f79734262c9ec2ba09d90aa9abc16de5
\
  zeek -C -r /pcaps/the-PCAP-file \
  LogAscii::use_json=T
```

No custom Wazuh decoder was written Zeek's single line JSON output is parsed automatically by Wazuh's built-in json decoder, exposing fields such as id.orig_h, func, and orig_pkts directly to the rules.

4.1 Rules and rationale

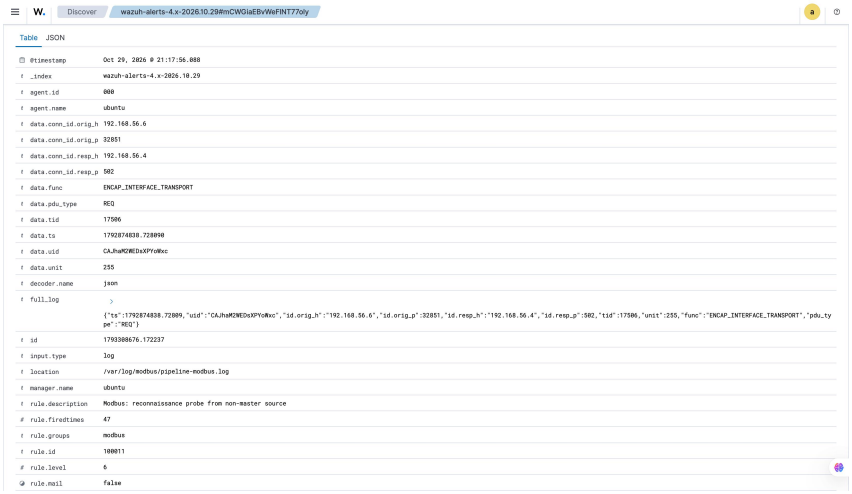
Rule	Level	Condition	Why it's reliable
100011	6	READ_INPUT_REGISTERS / ENCAP_INTERFACE_TRANSPORT, non-master source	Baseline never produces either, from any source
100012	12	WRITE_SINGLE_COIL, any source	Baseline has zero writes also catches a compromised master
100013	12	WRITE_SINGLE_REGISTER, any source	Same as above, for setpoints

Rule	Level	Condition	Why it's reliable
100014	10	PROGRAM_UNITY / DIAGNOSTICS	Neither code appears in normal operation
100017	9	func ends in _EXCEPTION	Baseline produces zero exceptions
100015/16	5/13	Non-master connection; escalates at 4+-digit packet count	Only the master's session shape is legitimate

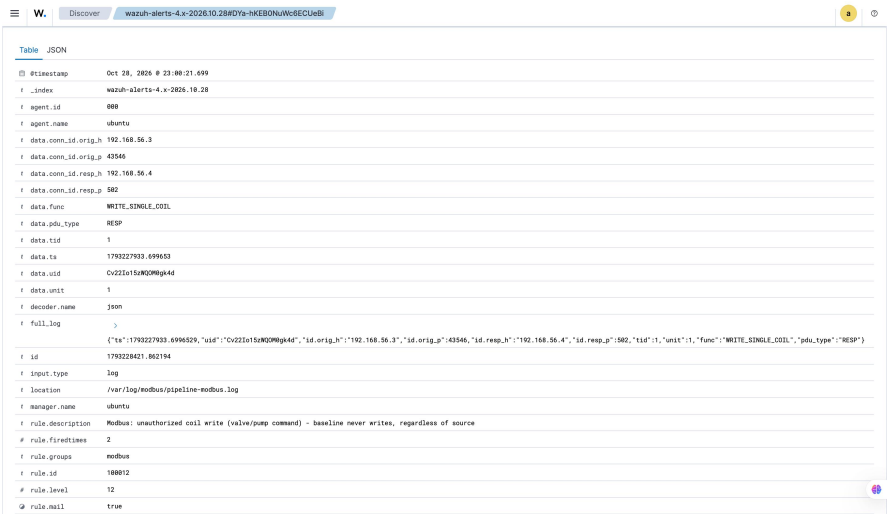
Rules 100012/100013 were deliberately left source-independent: the baseline proves the master never writes, so treating any write as anomalous closes a real gap (a compromised master) at a known cost, examined in Section 5.

4.2 Confirmed detections

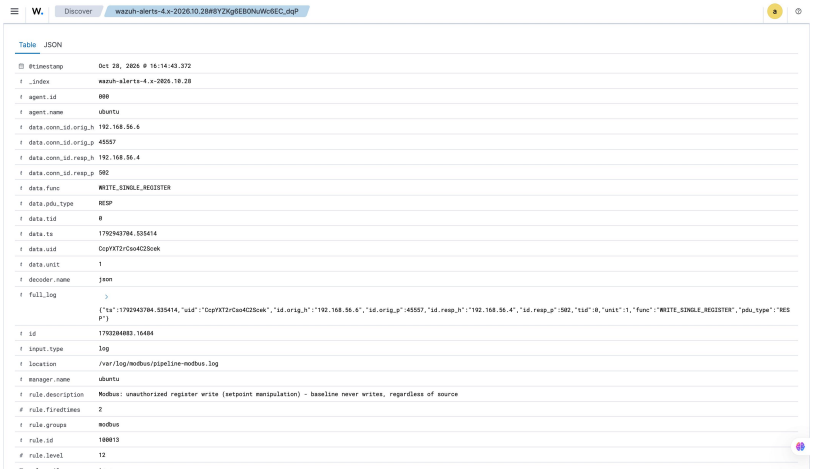
Each rule was tested by replaying its capture's Zeek log through the Wazuh pipeline and confirming (a) the alert fires on the attack, and (b) it stays silent on the baseline. Representative alerts:



Alert: Rule 100011, reconnaissance.



Alert: Rule 100012, coil write.



Alert: Rule 100014, diagnostic attempt.



Alert: Rule 100017, exception response.



Alert: Rule 100015, non-master connection.

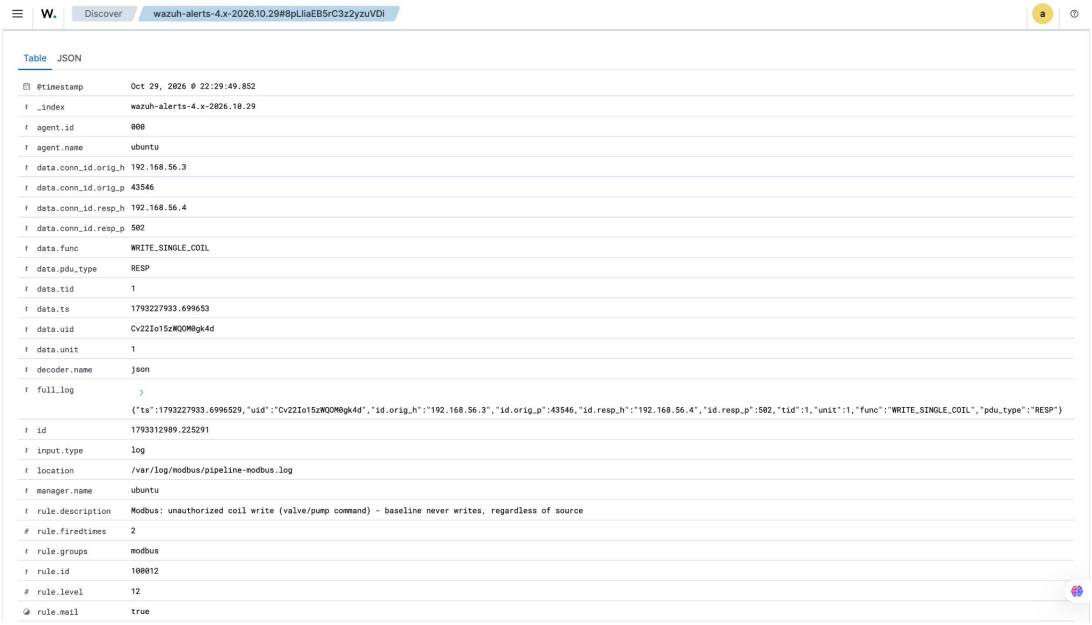


Alert: Rule 100016, high-volume connection (flood).

5. Edge Cases

5.1 False positive: legitimate write from the master

A WRITE_SINGLE_COIL sent from the master's own address (simulating an authorized engineer's manual valve operation) still triggers rule 100012 a direct, deliberate consequence of the source independent write rules.

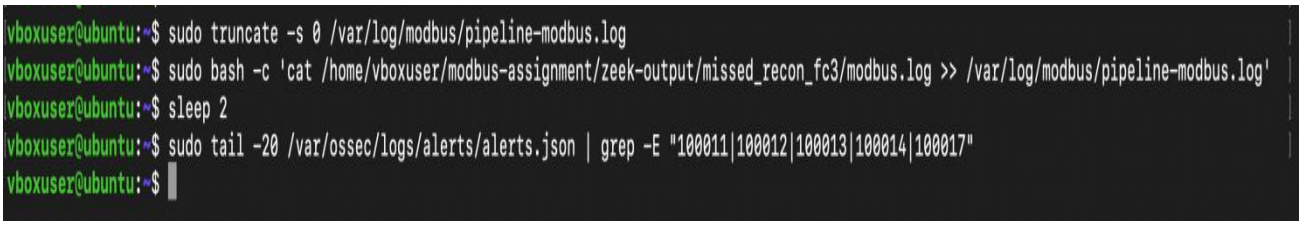


False positive: Rule 100012 fired on a master-originated write

Change proposed: correlate writes against a maintenance window allowlist, lowering severity inside approved windows. Cost: requires an external change management data source this lab lacks; absent that, the current tradeoff is intentional, more noise during legitimate operations in exchange for catching a compromised master.

5.2 Missed attack: reconnaissance using the baseline's own function code

A READ_HOLDING_REGISTERS request (FC3) from the attacker, interleaved with the master's own polling, produced no alert at all confirmed by an empty grep across every rule ID:



Pipeline test confirming silence on the FC3 recon variant.

Rule 100011 only flags the function codes observed during testing; an attacker reading with the same function code the master uses is indistinguishable from baseline traffic by function code alone. Change proposed: flag READ_HOLDING_REGISTERS specifically from any non master source, independent of function code. Cost: a hard dependency on a fixed source IP allowlist that silently breaks if the master's address ever changes.

6. Conclusion

Because the legitimate traffic pattern here is extremely narrow one source, one function code, one connection, a stable rate simple deviation-based rules give strong, explainable coverage with minimal false positive risk under normal conditions. The two edge cases show the honest limits of that approach: rules broad enough to catch a compromised trusted source will also flag legitimate use of that source, and rules tuned to the specific attacks tested can be evaded by an attacker who mimics legitimate traffic shape rather than deviating from it. Closing that second gap durably would require moving from function code/source matching toward value range validation and behavioral baselining a natural next step beyond this assignment's scope.